

CaesarPublisherV1

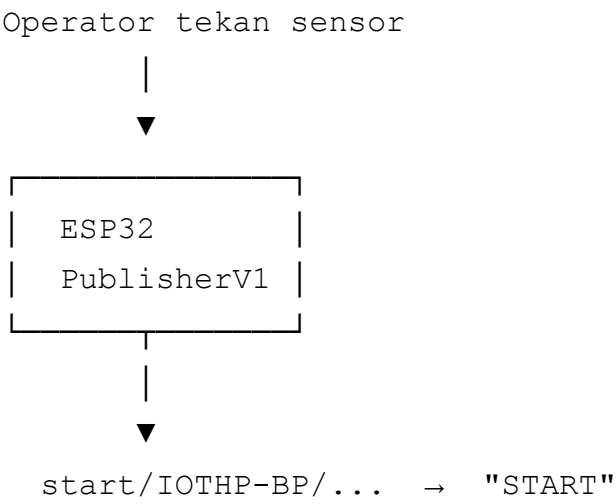
Hardware simulator untuk CaesarFirmwareV1. ESP32 ini mensimulasikan siklus mesin cetak tanpa mesin asli. Operator tekan tombol di sensor, firmware kirim sinyal START dan siklus selesai ke broker MQTT yang sama dengan simulator software.

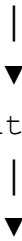
Perbedaan dengan Firmware Utama

Aspek	CaesarFirmwareV1	CaesarPublisherV1
Fungsi	Kontrol mesin cetak asli	Simulasi siklus via sensor
HMI Nextion	Ya	Tidak
Barcode scanner	Ya	Tidak
Quota/ISI tracking	Ya	Tidak
Operator/Mould/Lot	Ya	Tidak
Interlock GPIO25	Ya	Tidak
MQTT finish topic	Sama	Sama
MQTT broker	Sama	Sama

PublisherV1 mengirim data ke topik yang sama dengan simulator software (`start/...` dan `finish/...`). CaesarFirmwareV1 tidak tahu bedanya. Data dari hardware atau software sama saja.

Cara Kerja





Operator tekan up/down 5x

finish/IOTHP-BP/... →

`{"event": "1cycle", "startTime": "...", "finishTime": "..."}`

Sensor

Sensor	Pin	Fungsi
Front (depan)	GPIO32	Mulai siklus layer depan
Back (belakang)	GPIO35	Mulai siklus layer belakang
Up (naik)	GPIO33	Gerakan naik (step 1, 3, 5)
Down (turun)	GPIO25	Gerakan turun (step 2, 4)
WiFi LED	GPIO26	Indikator koneksi WiFi

File

config.h

Pengaturan utama. Semua bisa diubah tanpa menyentuh kode program.

Define	Value	Fungsi
WIFI_SSID	TIME0	SSID WiFi
WIFI_PASS	1234Saja	Password WiFi
MQTT_SERVER	broker.hivemq.com	Broker MQTT
MQTT_PORT	1883	Port broker
MQTT_ADDR	91c21b8614cd	ID device
MQTT_CLIENT_ID	CaesarPublisher/91c21b8614cd	Client ID MQTT
START_TOPIC	start/IOTHP-BP/91c21b8614cd	Topik sinyal START
FINISH_TOPIC	finish/IOTHP-BP/91c21b8614cd	Topik siklus selesai
CONTROL_TOPIC	CONTROL/IOTHP-BP/91c21b8614cd	Topik perintah

Define	Value	Fungsi
NTP_SERVER	pool.ntp.org	Server waktu
WIB_TIMEZONE	WIB-7	Zona waktu
FRONT_PIN	GPIO32	Sensor depan
BACK_PIN	GPIO35	Sensor belakang
UP_PIN	GPIO33	Sensor naik
DOWN_PIN	GPIO25	Sensor turun
WIFI_LED_PIN	GPIO26	LED WiFi
OTA_USERNAME	admin	Username update firmware
OTA_PASSWORD	123654789	Password update firmware

CaesarPublisherV1.ino

File utama yang menghubungkan semua bagian. Berisi objek global dan alur utama program.

Objek global:

- `WiFiClient wifiClient` – objek koneksi WiFi
- `PubSubClient mqttClient(wifiClient)` – objek koneksi MQTT
- `WebServer httpServer(80)` – web server di port 80 untuk OTA update
- `ESP32HTTPUpdateServer httpUpdater` – handler halaman web update firmware

setup() – dijalankan sekali saat ESP32 dinyalakan:

1. `Serial.begin(115200)` – nyalakan serial monitor untuk debug
2. `pinMode(FRONT_PIN, INPUT)` – set GPIO32 sebagai input (sensor depan)
3. `pinMode(BACK_PIN, INPUT)` – set GPIO35 sebagai input (sensor belakang)
4. `pinMode(UP_PIN, INPUT)` – set GPIO33 sebagai input (sensor naik)
5. `pinMode(DOWN_PIN, INPUT)` – set GPIO25 sebagai input (sensor turun)
6. `pinMode(WIFI_LED_PIN, OUTPUT)` – set GPIO26 sebagai output (LED WiFi)
7. `digitalWrite(WIFI_LED_PIN, LOW)` – matikan LED saat awal
8. `initCycleInputs()` – baca posisi awal semua sensor agar tidak false trigger
9. `initTime()` – set zona waktu WIB sebelum NTP jalan
10. `setupWifi()` – mulai koneksi WiFi
11. `setupMqtt()` – set alamat broker dan callback
12. `httpUpdater.setup(...)` – daftarkan halaman web update firmware di `/update`

13. `httpServer.begin()` – nyalakan web server di port 80

`loop()` – dijalankan ratusan kali per detik:

1. `mqttLoop()` – proses WiFi reconnect, MQTT reconnect, dan pesan masuk
 2. `syncTime()` – ambil waktu dari internet jika belum sinkron
 3. `httpServer.handleClient()` – handle request dari browser (untuk OTA update)
 4. `updateCycleInputs()` – baca semua sensor, proses logika siklus
-

wifi_mqtt.ino

Mengatur koneksi WiFi dan MQTT. Semua komunikasi dengan server melewati file ini.

Variabel global:

- `lastWifiAttempt` – timestamp kapan terakhir coba sambung WiFi. Digunakan untuk delay 5 detik agar ESP32 tidak spam reconnect.
- `lastMqttAttempt` – timestamp kapan terakhir coba sambung MQTT. Delay sama, 5 detik.

`mqttCallback(topic, payload, length)` – handler saat terima pesan dari server. Dipanggil otomatis oleh PubSubClient saat ada pesan masuk. Cek dua kondisi:

1. Pesan datang dari `CONTROL_TOPIC` DAN isinya `REBOOT` (6 karakter) → restart ESP32 via `ESP.restart()`
2. Selain itu → diabaikan

PublisherV1 hanya menerima satu jenis pesan: perintah reboot.

`setupWifi()` – mulai koneksi WiFi:

1. `WiFi.mode(WIFI_STA)` – set mode station (bukan access point)
2. `WiFi.setHostname(MQTT_CLIENT_ID)` – set nama device di jaringan
3. `WiFi.begin(WIFI_SSID, WIFI_PASS)` – mulai koneksi
4. `lastWifiAttempt = millis()` – catat waktu percobaan

`setupMqtt()` – konfigurasi MQTT:

1. `mqttClient.setServer(MQTT_SERVER, MQTT_PORT)` – set alamat broker
2. `mqttClient.setCallback(mqttCallback)` – daftarkan handler pesan masuk
3. `mqttClient.setBufferSize(256)` – buffer untuk payload (cukup untuk JSON siklus)
4. `mqttClient.setKeepAlive(60)` – jika tidak ada aktivitas 60 detik, koneksi putus

`reconnectWifi()` – auto-reconnect WiFi:

1. Jika WiFi sudah terhubung → skip
2. Jika belum dan sudah lewat 5 detik sejak percobaan terakhir → putus WiFi, sambung ulang
3. Delay 5 detik mencegah ESP32 mencoba sambung ulang terlalu sering

`reconnectMqtt()` – auto-reconnect MQTT:

1. Jika MQTT sudah terhubung → skip
2. Jika belum dan sudah lewat 5 detik → coba `mqttClient.connect()`
3. Setelah terhubung → subscribe `CONTROL_TOPIC` agar bisa terima perintah REBOOT

`mqttLoop()` – fungsi utama yang dipanggil dari `loop()` :

1. `reconnectWifi()` – coba sambung WiFi jika perlu
 2. `digitalWrite(WIFI_LED_PIN, ...)` – nyalakan LED jika WiFi terhubung, matikan jika tidak
 3. Jika WiFi belum terhubung → skip (tidak bisa proses MQTT tanpa WiFi)
 4. `reconnectMqtt()` – coba sambung MQTT jika perlu
 5. Jika MQTT terhubung → `mqttLoop()` untuk proses pesan masuk
-

time.ino

Pengambilan waktu dari internet (NTP). Waktu digunakan untuk timestamp di payload siklus.

Variabel global:

- `ntpConfigured` – apakah sudah pernah jalankan `configTzTime()` . Cukup sekali.
- `ntpReady` – apakah waktu dari NTP sudah berhasil diambil
- `lastNtpAttempt` – kapan terakhir coba ambil waktu
- `NTP_RETRY_INTERVAL` – interval retry 1 detik

`initTime()` – panggilan sekali saat `setup()` :

1. `setenv("TZ", WIB_TIMEZONE, 1)` – set variabel environment zona waktu ke "WIB-7"
2. `tzset()` – beritahu sistem bahwa semua waktu harus dikonversi ke WIB

Tidak perlu internet untuk fungsi ini. Waktu belum tersedia – hanya zona waktu yang diatur.

`syncTime()` – sinkronisasi waktu dari NTP. Dipanggil terus-menerus dari `loop()` :

1. Jika waktu sudah ready → skip (sudah sinkron)
2. Jika WiFi belum terhubung → skip (butuh internet)

3. Jika belum lewat 1 detik sejak percobaan terakhir → skip (delay)
4. Jalankan `configTzTime(WIB_TIMEZONE, NTP_SERVER)` sekali pertama
5. Coba ambil waktu dari server via `getLocalTime()`
6. Jika berhasil → `ntpReady = true`

Setelah `ntpReady = true`, waktu tersedia untuk semua fungsi yang membutuhkan timestamp.

`getTimestamp(buffer, length)` – ambil waktu saat ini dan format ke ISO8601 WIB:

1. Jika NTP belum ready → return `false`
2. Ambil waktu dari sistem via `getLocalTime()`
3. Format ke string: `YYYY-MM-DDTHH:MM:SS.000+07:00`
4. Return `true` jika berhasil

Contoh output: `2026-07-17T14:30:00.000+07:00`

Waktu tidak tersedia sampai WiFi terhubung dan NTP sinkron (biasanya 2-5 detik setelah WiFi nyambung).

cycle.ino

Logika siklus dari sensor. File ini menentukan kapan siklus dimulai, bagaimana menghitung gerakan, dan kapan siklus selesai.

Variabel state:

- `frontLatched` / `backLatched` / `upLatched` / `downLatched` – posisi terakhir tiap sensor. Digunakan untuk mendeteksi tepi naik (sensor baru saja ditekan, bukan sedang ditekan terus).
- `cycleStarted` – apakah siklus sedang berjalan
- `expectUp` – gerakan selanjutnya harus up (mencegah double count)
- `stepCount` – jumlah gerakan valid yang terhitung
- `cycleStartTime` – waktu saat siklus dimulai (dari NTP)

`initCycleInputs()` – panggilan sekali saat `setup()` :

Baca posisi awal semua sensor. Tanpa ini, jika sensor sudah ditekan saat ESP32 nyala, firmware akan false trigger siklus baru di detik pertama.

`publishStart()` – kirim sinyal START ke broker:

1. Cek MQTT connected. Jika tidak → skip.
2. Publish `"START"` tanpa retained ke `START_TOPIC`

3. Pesan ini memberitahu CaesarFirmwareV1 bahwa siklus baru dimulai

publishCycle() – kirim JSON siklus selesai:

1. Cek MQTT connected. Jika tidak → skip.
2. Ambil waktu sekarang dari NTP via `getTimestamp()`
3. Buat JSON: `{"event":"1cycle","startTime":"...","finishTime":"..."}`
4. `startTime` = waktu saat siklus dimulai (dari `cycleStartTime`)
5. `finishTime` = waktu saat ini (waktu siklus selesai)
6. Publish tanpa retained ke `FINISH_TOPIC`

CaesarFirmwareV1 terima JSON ini → increment siklus, kurangi kuota, update layar, kirim data ke server.

startCycle() – mulai siklus baru:

1. Jika siklus sudah berjalan (`cycleStarted == true`) → skip (tidak mulai siklus baru sebelum yang sebelumnya selesai)
2. Ambil waktu dari NTP. Jika gagal → skip (timestamp tidak tersedia)
3. Set `cycleStarted = true`
4. Reset `stepCount = 0`
5. Set `expectUp = true` (gerakan pertama harus up)
6. Simpan waktu mulai ke `cycleStartTime`
7. `publishStart()` – kirim sinyal START

finishCycle() – selesaikan siklus:

1. `publishCycle()` – kirim JSON siklus selesai
2. Set `cycleStarted = false`
3. Reset `stepCount = 0`
4. Set `expectUp = true`

updateCycleInputs() – fungsi utama yang dipanggil dari `loop()`. Membaca semua sensor dan memproses logika siklus:

1. Baca posisi semua sensor: `digitalRead(FRONT_PIN) == LOW` artinya sensor ditekan
2. Deteksi tepi naik: sensor baru saja ditekan (belum ditekan sebelumnya)
 - `frontPressed && !frontLatched` → sensor depan baru ditekan
 - `backPressed && !backLatched` → sensor belakang baru ditekan
3. Jika Front/Back baru ditekan → `startCycle()`
4. Deteksi gerakan up: sensor up ditekan, sebelumnya tidak, `expectUp == true`, dan siklus berjalan

- `stepCount++`, `expectUp = false`

5. Deteksi gerakan down: sensor down ditekan, sebelumnya tidak, `expectUp == false`, dan siklus berjalan

- `stepCount++`, `expectUp = true`

6. Update posisi terakhir semua sensor (untuk deteksi tepi di loop berikutnya)

7. Jika `stepCount >= 5` dan siklus berjalan → `finishCycle()`

Pola gerakan:

Sensor Front/Back ditekan → START

Up (step 1) → `expectUp = false`

Down (step 2) → `expectUp = true`

Up (step 3) → `expectUp = false`

Down (step 4) → `expectUp = true`

Up (step 5) → FINISH

Total 5 gerakan: 3 up + 2 down (atau 2 up + 3 down, tergantung urutan awal).

Topik MQTT

Topik	Arah	Isi
<code>start/IOTHP-BP/91c21b8614cd</code>	Publisher → Broker	"START"
<code>finish/IOTHP-BP/91c21b8614cd</code>	Publisher → Broker	JSON siklus
<code>CONTROL/IOTHP-BP/91c21b8614cd</code>	Broker → Publisher	"REBOOT"

OTA Update

Akses `http://[IP_ESP32]/update` dari browser. Login:

- Username: `admin`
- Password: `123654789`

Upload file `.bin` firmware baru dari browser. ESP32 akan restart otomatis setelah upload selesai.

Troubleshooting

Masalah	Solusi
---------	--------

Masalah	Solusi
LED WiFi mati	WiFi tidak terhubung. Cek SSID dan password di <code>config.h</code>
Siklus tidak mulai	Pastikan sensor Front/Back ditekan dulu
Siklus tidak selesai	Hitung 5 gerakan: up, down, up, down, up
Data tidak sampai ke firmware	Cek koneksi internet, pastikan broker.hivemq.com:1883 bisa diakses
Waktu salah di payload	NTP belum sinkron. Tunggu beberapa detik setelah WiFi terhubung